

## **Sistema inteligente de detección de espacios de estacionamiento**

### **1. Descripción del problema**

El estacionamiento cuenta con una gran cantidad de espacios disponibles, por lo que el personal encargado no puede conocer visualmente y en todo momento qué lugares están ocupados y cuáles están disponibles.

Esto provoca:

- Dificultad para localizar espacios libres.
- Pérdida de tiempo recorriendo el estacionamiento.
- Mayor carga de trabajo para el personal.
- Falta de información en tiempo real.
- Posibles errores humanos al determinar la disponibilidad.

### **Solución propuesta**

Desarrollar un sistema inteligente capaz de utilizar cámaras de video y técnicas de Inteligencia Artificial para identificar automáticamente el estado de cada espacio:

- **Libre**
- **Ocupado**

La información será procesada y mostrada en una interfaz para que el encargado pueda conocer el estado del estacionamiento sin necesidad de recorrerlo físicamente.

### **2. Lenguaje de programación**

#### **Python**

#### **Lenguaje seleccionado: Python**

Python sería nuestra primera opción debido a que cuenta con un ecosistema muy amplio para **Inteligencia Artificial, Machine Learning y Computer Vision**.

#### **Principales ventajas**

- Sintaxis sencilla y rápida de desarrollar.
- Amplio soporte para Inteligencia Artificial.
- Gran cantidad de bibliotecas de visión artificial.
- Integración sencilla con modelos de Machine Learning y Deep Learning.
- Excelente soporte para procesamiento de imágenes y video.

- Facilidad para crear APIs y servicios backend.

#### Bibliotecas que podrían utilizarse

| Biblioteca  | Función                           |
|-------------|-----------------------------------|
| OpenCV      | Procesamiento de imágenes y video |
| YOLO        | Detección de objetos              |
| PyTorch     | Deep Learning                     |
| NumPy       | Procesamiento numérico            |
| FastAPI     | API para comunicar el sistema     |
| Ultralytics | Implementación de modelos YOLO    |

### 3. Inteligencia Artificial

Para la inteligencia artificial utilizaría **YOLO**

YOLO es una familia de modelos de **detección de objetos en tiempo real**, por lo que resulta apropiada para analizar las imágenes provenientes de las cámaras.

El modelo podría entrenarse o configurarse para identificar vehículos y determinar si estos se encuentran dentro de un espacio de estacionamiento.

#### ¿Cómo funcionaría?

La cámara obtiene una imagen:

**Cámara → Imagen → IA → Detección del vehículo → Estado del espacio**

Por ejemplo:

Espacio 01 → 🚗 → OCUPADO

Espacio 02 → → LIBRE

Espacio 03 → 🚗 → OCUPADO

Espacio 04 → → LIBRE

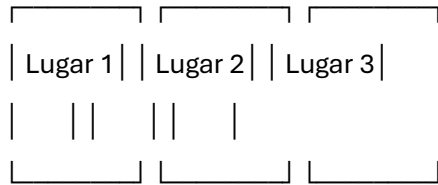
El sistema procesaría continuamente las imágenes para actualizar el estado.

### 4. ¿Cómo determinaríamos si un lugar está ocupado?

Primero necesitamos conocer dónde están físicamente los espacios del estacionamiento.

Por eso propondría utilizar **zonas de interés (ROI — Region of Interest)**.

Cada espacio tendría una zona previamente definida:



La IA detectaría los vehículos y el sistema comprobaría si el vehículo se encuentra dentro de alguna de esas zonas.

Por ejemplo:

Vehículo detectado



¿Interseca con ROI?



Sí



Lugar ocupado

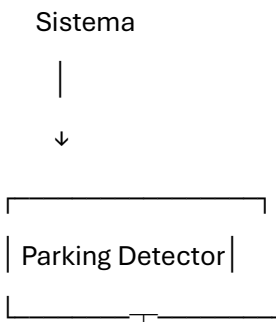
Esto permite evitar que un vehículo que simplemente está circulando por el estacionamiento sea considerado como un vehículo estacionado.

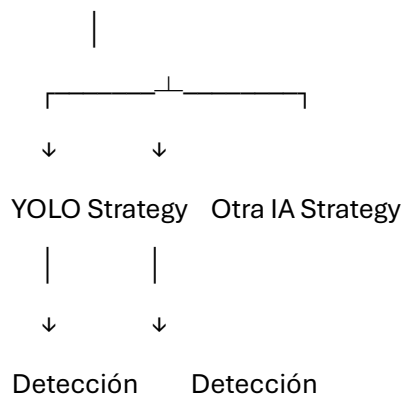
## 5. Patrón de diseño

### Patrón Strategy

El **Strategy Pattern** sería bastante útil porque permite cambiar el algoritmo utilizado para determinar si un espacio está ocupado sin modificar el resto del sistema.

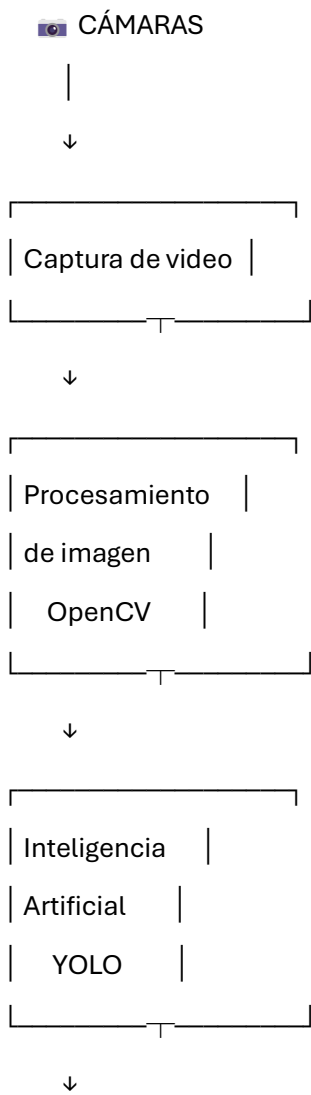
Por ejemplo:

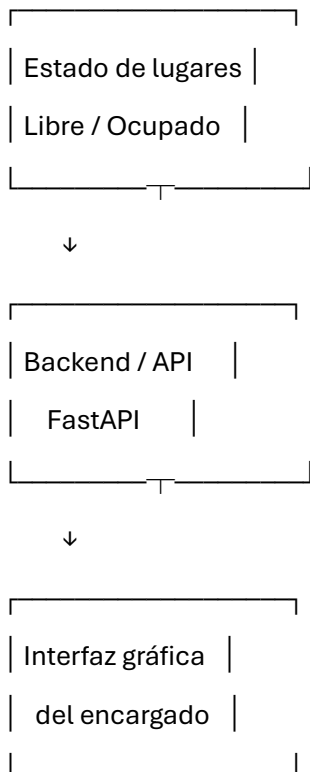




## 6. Arquitectura propuesta

Utilizar una arquitectura dividida en varias capas:





## 7. Base de datos

Utilizar una base de datos.

### PostgreSQL

Porque permitiría almacenar información como:

- Identificador del estacionamiento.
- Número del espacio.
- Estado actual.
- Cámara asociada.
- Ubicación.
- Fecha y hora de actualización.

Ejemplo conceptual:

| ID | Espacio | Cámara | Estado  |
|----|---------|--------|---------|
| 1  | A-01    | CAM-01 | Ocupado |

| ID | Espacio | Cámara | Estado  |
|----|---------|--------|---------|
| 2  | A-02    | CAM-01 | Libre   |
| 3  | A-03    | CAM-01 | Ocupado |
| 4  | A-04    | CAM-01 | Libre   |

## 8. Metodología de desarrollo

Para desarrollar el proyecto utilizaría **Scrum** como metodología Agile.

Esto es importante porque el proyecto puede dividirse perfectamente en funcionalidades pequeñas.

### Sprint 1 — Análisis

Definir:

- Problema.
- Requerimientos.
- Número de cámaras.
- Cantidad de espacios.
- Arquitectura.
- Tecnología.

### Sprint 2 — Captura de video

Implementar:

- Conexión con cámaras.
- Captura de imágenes.
- Procesamiento básico con OpenCV.

### Sprint 3 — Inteligencia Artificial

Implementar:

- Modelo YOLO.
- Detección de vehículos.
- Configuración de confianza.
- Pruebas de detección.

## **Sprint 4 — Detección de espacios**

Implementar:

- Definición de ROI.
- Asociación vehículo ↔ espacio.
- Determinación de Libre/Ocupado.

## **Sprint 5 — Backend**

Implementar:

- API.
- Consulta de estados.
- Actualización de información.
- Comunicación con la IA.

## **Sprint 6 — Interfaz**

Crear un panel donde el policía pueda visualizar:

ESTACIONAMIENTO

 A-01     A-02  
 A-03     A-04  
 A-05     A-06

LIBRES: 4

OCUPADOS: 2

## **Sprint 7 — Pruebas**

Realizar pruebas en diferentes condiciones:

- Día.
- Noche.
- Lluvia.
- Diferentes ángulos.
- Vehículos grandes.
- Vehículos pequeños.

- Oclusiones.
- Alta cantidad de vehículos.

## 9. Tecnologías propuestas

La documentación podría quedar resumida así:

| Componente        | Tecnología                             |
|-------------------|----------------------------------------|
| Lenguaje          | <b>Python</b>                          |
| IA                | <b>YOLO</b>                            |
| Visión artificial | <b>OpenCV</b>                          |
| Deep Learning     | <b>PyTorch</b>                         |
| Backend           | <b>FastAPI</b>                         |
| Base de datos     | <b>PostgreSQL</b>                      |
| Arquitectura      | <b>API REST</b>                        |
| Patrón de diseño  | <b>Strategy</b>                        |
| Metodología       | <b>Scrum / Agile</b>                   |
| Cámaras           | IP / CCTV                              |
| Frontend          | React + TypeScript ( <i>opcional</i> ) |

## 10. Flujo general del sistema

Finalmente, el funcionamiento completo sería:

 Cámara

↓

Captura video

↓

OpenCV

↓



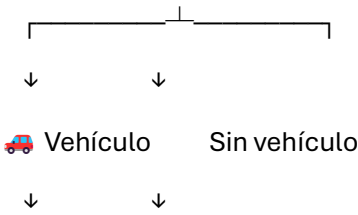
Modelo YOLO / IA



Detectar vehículos



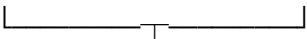
Comparar con espacios



 Vehículo                      Sin vehículo



OCUPADO                      LIBRE



Base de datos



API REST



Panel del policía